

Répartition des tâches — Traçabilité IA

Projet en 3 parties : **backend** (Spring Boot) · **frontend** (Angular) · **ml-service** (Flask / scikit-learn). Équipe : **A = Yahya** · **B = binôme**.

Principe : découpage vertical. Chacun prend une **fonctionnalité complète de bout en bout** (son propre morceau de backend + ml-service + frontend). Les deux verticales sont indépendantes → chacun travaille **à son rythme, sans attendre l'autre**, tout en touchant les 3 dossiers.

Stack technique

- **Backend** : Spring Boot (Java 21) + PostgreSQL + Spring Security / JWT — *squelette à construire*.
- **Frontend** : Angular 21 (standalone, SSR) — *landing + login faits*, à connecter au backend.
- **ml-service** : Flask + scikit-learn — *fonctionnel* (`/predict` , `/train` , `/domains` , `/health`), à enrichir.

Règles de travail

- **1 tâche = 1 responsable** : une seule personne code ET teste sa tâche avant de la partager.
- **Auto-test obligatoire** avant de dire « fini » : test unitaire + test réel (Docker / navigateur / Postman), documenté.
- **Rétro d'erreurs** après chaque tâche (modèle en bas).
- **Rotation** : chacun touche backend + frontend + ml-service.

Étape 0 — Fondation commune (à faire ensemble, ~1-2 jours)

Base partagée incontournable (sinon conflits). À faire **en binôme** une seule fois, puis chacun devient autonome.

Fondation commune (Sprint 0)	Dossiers
Config PostgreSQL + entité Utilisateur + Auth JWT + <code>/api/auth/login</code> + connexion login frontend	backend + frontend

Les 2 verticales indépendantes

Verticale A — Yahya

Décisions & Traçabilité

Tâche	Dossier
Entité <code>Decision</code> + repository + <code>DecisionController</code> / <code>Service</code> (CRUD)	backend
<code>MLDecisionService</code> : appelle <code>m1-service:5000/predict</code>	backend
Hash-chain (audit trail immuable)	backend
Améliorer <code>/predict</code> + explicabilité (facteurs/poids)	ml-service
Page « Nouvelle décision » (formulaire → ML)	frontend
Page « Décisions » (liste + détail + explicabilité)	frontend
Page « Validation humaine »	frontend

Verticale B — Binôme

Comparaison d'agents & Dashboard

Tâche	Dossier
Entité <code>Agent</code> + repository + endpoint <code>/api/comparaison</code> (classement %)	backend
Endpoint <code>/api/stats</code> (chiffres du dashboard)	backend
Multi-modèles (plusieurs agents) + endpoint <code>/compare</code>	ml-service
Métriques (accuracy / consensus) dans le ML	ml-service
Page « Comparaison agents » (tableau + graphique %)	frontend
Page « Dashboard » (statistiques)	frontend

A touche les 3 dossiers. B touche les 3 dossiers. Et ils sont indépendants.

Workflow git (éviter les conflits)

- **Chacun sa branche** : `feature/decisions` (A) et `feature/comparaison` (B).
- **Petits commits fréquents** + `git pull` régulier de `main`.
- **Pull Requests** pour fusionner dans `main` (l'autre relit).
- Fichiers partagés (`SecurityConfig.java` , `application.properties` , `app.routes.ts` , `pom.xml`) : se prévenir avant d'y toucher.

Tableau de suivi

Statuts :  À faire ·  En cours ·  En test ·  Fini ·  Bloqué

Tâche	Dossier	Responsable	Statut	Erreurs rencontrées	Solution
Fondation login	backend+frontend	A + B		—	—
Entité Decision	backend	A		—	—
Multi-modèles ML	ml-service	B		—	—
...	...	...	...	...	...

Modèle de « rétro d'erreurs » (après chaque tâche)

```
### [Tâche] – [Responsable] – [Date]
- Ce qui a été fait :
```

- Erreurs rencontrées :
- Comment résolu :
- Conseil pour l'autre :
- Comment j'ai testé : (unitaire + réel)